Exactly-Once Processing: Claims and Limits

what 'exactly-once' really promises

Watch the 4-min video

In one line: no network can deliver every message exactly once, but you can get exactly-once *effects*: at-least-once delivery plus idempotent or transactional processing; Kafka's exactly-once covers read-process-write inside Kafka, and anything outside it still needs idempotency keys or deduplication.

1 The vendor's promise

A vendor promises **exactly-once delivery**: every message delivered once, never lost, never duplicated. It sounds perfect. But over an unreliable network with machines that crash, a sender can never know whether a message whose acknowledgement was lost arrived or not (lesson 2). It must either resend (risking a duplicate) or not (risking a loss).

2 Three delivery guarantees

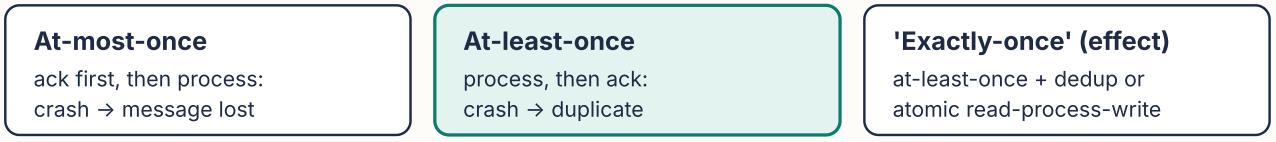


Figure 1. The order of 'process' and 'ack' decides between loss and duplicates. Exactly-once is built on top of at-least-once.

So the honest version is **exactly-once processing**, sometimes called **effectively-once**: messages may be delivered more than once, but their **effect** is applied once. Two ways to get it: make processing **idempotent** (lesson 20), or commit the output and the input position **atomically**, so either both happen or neither does.

3 How Kafka does it

Kafka combines three features: the **idempotent producer** (the broker drops duplicate retries using producer ID and sequence numbers), **transactions** (writes to several partitions commit or abort together), and committing the consumer's **offsets inside the same transaction**. Consumers that read with `isolation.level=read_committed` never see aborted output.

WORKED EXAMPLE: MERCHANT TOTALS WITH KAFKA STREAMS

A Kafka Streams app reads payment events, aggregates totals per merchant and writes results to an output topic. With `processing.guarantee=exactly_once_v2`, each batch's output records, state-store changelog updates and input offsets commit in one transaction. If the app crashes mid-batch, the transaction aborts, the batch is re-read from the last committed offsets, and the totals are not double counted. But if the same app also sent an email per payment, a crash could still send that email twice: the email server isn't part of Kafka's transaction.

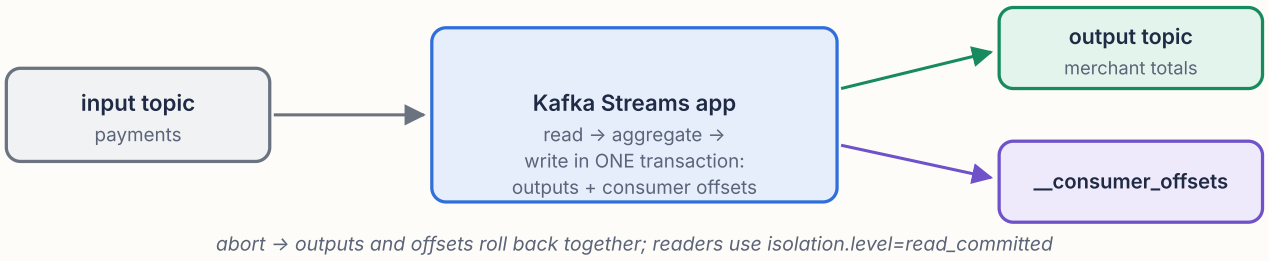


Figure 2. Outputs and input offsets in one transaction: the heart of Kafka's exactly-once.

4 Where the guarantee ends

Exactly-once holds only within the system that controls both the input position and the output. Writing to Postgres, calling a card API or sending email sits outside. There you need idempotency keys, a dedup table, or storing the input offset in the same database transaction as the result (then resuming from that stored offset).



Figure 3. Exactly-once inside one system; idempotency at its edges.

COMMON MISTAKES

- Believing 'exactly-once' in a product means no duplicates anywhere.
- Turning on Kafka EOS and calling external APIs without idempotency keys.
- Downstream consumers reading with read_uncommitted and seeing aborted data.
- Paying EOS latency costs where idempotent upserts would be simpler.

5 Key terms

At-most-once: never duplicated, may be lost

At-least-once: never lost, may be duplicated

Exactly-once processing: the effect is applied once despite redelivery

Idempotent producer: broker drops duplicate retries

Kafka transaction: atomic writes to many partitions plus offsets

read_committed: consumers skip aborted records

6 Interview questions

Q1. Is exactly-once delivery possible? BASIC

Not over an unreliable network in general: a sender can't distinguish a lost message from a lost ack. Exactly-once effects are possible with retries plus deduplication or atomic commits.

Q2. How does Kafka provide exactly-once semantics? BASIC

Idempotent producers, transactions across partitions, and committing consumer offsets in the same transaction; consumers read only committed data.

Q3. Your Kafka consumer writes to Postgres. How do you avoid double writes after a crash? SCENARIO

Store the processed offset (or message ID) in the same Postgres transaction as the data and resume from it, or use idempotent upserts keyed by event ID.

Q4. A team enabled exactly-once in Kafka Streams but customers still get duplicate emails. Why? SCENARIO

The email send is outside Kafka's transaction. Make it idempotent, for example by recording sent notifications with a unique key before sending.

Q5. What's the cost of Kafka transactions? TRICKY

Extra latency (commit markers, coordinator round trips), lower throughput for small batches, and consumers waiting for open transactions before reading past them.

Q6. Why do people say 'effectively-once' instead? TRICKY

Because messages may still be delivered and processed several times internally; only the observable effect happens once.

Next: SD-H-022 · Linearizability and Serializability backendroadmap.com